SmartReply Agent — Document technique

Version 1.0 · 5 novembre 2026 · Koffa Jean Agoudavi

Périmètre	État actuel vérifié du dépôt, sans projection de roadmap.
Référence	Commit 554f022 · tête Alembic a9c4e7f2b6d1.
Diffusion	Document partageable ; aucun secret ni identifiant externe.

1. Pile technique en un coup d’œil

SmartReply Agent est un SaaS multi-utilisateur qui analyse les emails professionnels, classe les demandes, alimente un Google Sheet propre à chaque entreprise et prépare un brouillon Gmail. L’envoi n’est jamais automatique : la décision reste à l’utilisateur, dans Gmail ou via une action Telegram explicitement déclenchée lorsque cette intégration optionnelle est activée.

Le système est organisé autour d’une application FastAPI unique, qui sert à la fois les API et plusieurs interfaces statiques en HTML, CSS et JavaScript natifs. PostgreSQL/Neon porte le contrôle, l’authentification et l’idempotence ; Gmail et Google Sheets restent le plan de données métier. Ce document décrit strictement l’état du dépôt au commit de référence 554f022, sans supposer l’état réel de Render, Neon ou des services externes.

Couche	Technologie	Rôle actuel
Interfaces	HTML5, CSS3, JavaScript natif	Accueil, dashboard, administration, contact et pages légales ; aucun build front-end.
API	Python 3.11, FastAPI, Pydantic	Authentification, entreprises, OAuth, dashboard, administration, contact, pipeline et webhooks.
Orchestration	Services Python modulaires	Chaîne Gmail → filtre → Gemini → Sheets → brouillon → Telegram facultatif.
Données	PostgreSQL/Neon, Google Sheets	Contrôle et idempotence dans Neon ; données métier isolées dans un Sheet par entreprise.
Intégrations	Gmail, Drive, Sheets, Gemini, Telegram, SMTP	OAuth utilisateur, analyse IA, stockage métier, notifications facultatives et formulaire Contact.
Livraison	Render, Alembic, GitHub Actions	Migration au démarrage, service Uvicorn, lint, compilation, tests et contrôle de la tête Alembic.

1.1 Repères quantitatifs du dépôt

Indicateur	Valeur vérifiée	Portée
Opérations OpenAPI	42	41 chemins en environnement de développement ; documentation désactivée en production.
Tables SQLAlchemy	10	Utilisateurs, entreprises, OAuth, Telegram, traitement, activité et administration.
Migrations Alembic	12	Chaîne unique ; tête a9c4e7f2b6d1.
Tests automatisés	288	Suite unittest complète exécutée avec succès le 5 novembre 2026.
Modules Python applicatifs	71 fichiers	Sous app/, hors migrations, scripts et tests.
Périmètre UI	6 surfaces	Accueil, dashboard, admin, contact, CGU et confidentialité.

1.2 Flux principal

```
Cron externe
-> POST /api/v1/pipeline/run (X-CRON-SECRET)
-> verrou PostgreSQL + sélection équitable des entreprises
-> Gmail : messages reçus après l'activation
-> filtrage anti-spam / anti-newsletter
-> réservation idempotente dans processed_emails
-> Gemini : catégorie, priorité, résumé et proposition de réponse
-> Google Sheet « Emails »
-> brouillon Gmail dans le fil d'origine
-> Google Sheet « Brouillons »
-> notification Telegram facultative
```

Principe de contrôle humain

Le dashboard utilisateur ne réalise aucun envoi direct. Il permet de consulter les informations disponibles, d'ouvrir Gmail et d'ignorer un brouillon après confirmation. Un message d'administration est présenté uniquement sous forme de bannière non lue, sans historique utilisateur.

2. Application

2.1 Socle

Outil	Version déclarée	Usage
Python	3.11 dans la CI	Langage du backend, des migrations, des scripts et des tests.
FastAPI	>= 0.110	API REST, dépendances d'authentification, validation et service des surfaces statiques.
Uvicorn	>= 0.28	Serveur ASGI lancé sur Render avec --no-proxy-headers.
Pydantic	>= 2.7	Schémas d'entrée/sortie et validation des réglages.
SQLAlchemy	>= 2.0.25, < 2.1	Modèles, sessions, requêtes tenant-scoped et transactions.
Alembic	>= 1.13	Migrations versionnées de PostgreSQL.
HTML/CSS/JavaScript	Natif	Interfaces servies directement ; aucune dépendance front-end ni étape de compilation.

2.2 Bibliothèques et protocoles

Besoin	Composant	Fonction
Mots de passe	argon2-cffi	Hachage Argon2id ; aucun mot de passe en clair n'est conservé.
Sessions API	PyJWT / HS256	JWT signé contenant l'identifiant interne, l'email, iat et exp ; durée par défaut 24 h.
Secrets OAuth	cryptography / Fernet	Chiffrement des refresh tokens Gmail et Drive avant stockage.
Appels HTTP	httpx	Requêtes vers Google, Gemini et Telegram avec gestion d'erreurs bornée.
Google Sheets	gspread + google-auth	Ouverture du classeur d'entreprise via compte de service.
Validation email	email-validator	Normalisation et validation des adresses d'authentification et de contact.
Limitation anti-abus	redis-py	Compteurs Redis atomiques avec repli mémoire par processus.
Contact	SMTP SSL	Transmission authentifiée du formulaire public ; paramètres fournis par variables d'environnement.

2.3 Organisation de la logique

- app/main.py assemble les routeurs, applique le middleware de sécurité, sert les pages statiques et expose /health et /ready.
- app/modules/<domaine>/ sépare généralement router.py, schemas.py, models.py, service.py et les dépendances de sécurité propres au domaine.
- pipeline_selection prépare un passage équitable, applique l'allowlist et la suspension, puis prend un verrou transactionnel PostgreSQL.
- pipeline_execution charge un contexte d'entreprise isolé et orchestre les appels externes sans partager de token, de Sheet ou de chat entre tenants.
- processed_emails centralise les réservations, tentatives, états du brouillon, reprises et marqueurs de notification.
- Les interfaces app/home, app/dashboard, app/admin, app/contact, app/conditions et app/confidentialite sont autonomes et utilisent uniquement des ressources locales.

2.4 Surfaces fonctionnelles livrées

Surface	Accès	Fonction technique
Accueil	/	Page publique, présentation du pilote et orientation vers le dashboard ou le contact.
Dashboard	/dashboard/	JWT, onboarding, intégrations, indicateurs, emails, brouillons, paramètres, suspension et bannière d'information.
Administration	/admin/	JWT + allowlist + code secondaire ; vues opérationnelles, bannière interne et suspension du pipeline.
Contact	/contact/	Formulaire public borné, rate-limité et transmis par SMTP authentifié.
CGU	/conditions/	Page publique statique.
Confidentialité	/confidentialite/	Page publique statique.

3. Données, pipeline et isolation

3.1 Répartition du stockage

Emplacement	Données	Garantie principale
PostgreSQL / Neon	Comptes, entreprises, OAuth chiffré, liaisons Telegram, registre de traitement, activité, messages admin et audit.	Transactions, contraintes, unicité et filtrage par company_id.
Google Sheet d'entreprise	Onglets Emails et Brouillons, créés dans le Drive du client pour les nouvelles entreprises.	Séparation métier visible par entreprise ; accès writer du compte de service.
Gmail	Messages source, fils et brouillons créés par SmartReply.	Le brouillon est créé dans la boîte de l'utilisateur ; aucun corps d'email n'est stocké dans Neon.
Redis facultatif	Compteurs temporaires login, inscription, contact et déverrouillage admin.	TTL et incréments atomiques entre instances ; aucune donnée métier.
Navigateur	JWT dans localStorage ; brouillon d'onboarding dans sessionStorage.	Aucun refresh token Google ni code administrateur n'est envoyé ou conservé côté client.

3.2 Séquence de traitement par entreprise

Étape	Traitement	Protection
1	Sélectionner Gmail connecté + Sheet accessible ; exclure les suspensions actives.	Ordre par dernier passage, plafond par entreprise et budget global.
2	Intersecter avec PIPELINE_ALLOWED_COMPANY_IDS.	Les entreprises non allowlistées restent en simulation ; l'allowlist n'est pas une table commerciale.
3	Fixer pipeline_activated_at avant le premier appel externe.	Les emails antérieurs à cette coupure sont exclus des indicateurs.
4	Lister Gmail avec in:inbox after:<epoch>, puis vérifier internalDate.	Aucune modification du statut lu/non lu.
5	Réserver le message dans processed_emails.	Unicité (company_id, gmail_message_id), reprise contrôlée et trois tentatives maximum.
6	Filtrer, tronquer le corps et appeler Gemini avec le profil d'entreprise.	Le contenu est traité comme donnée non fiable ; le contexte est borné et isolé.
7	Écrire Emails, créer le brouillon Gmail, enregistrer son ID, puis écrire Brouillons.	Upserts par identifiant interne ; l'ID du brouillon est persisté avant les écritures suivantes.
8	Notifier Telegram si disponible et connecté.	Échec Telegram indépendant ; aucun nouvel appel Gemini ni nouveau brouillon lors d'une reprise de notification.

3.3 Idempotence et reprise

- Les états du registre sont processing, done, skipped, excluded et failed ; attempts est contraint entre 0 et 3.
- Un traitement done, skipped ou excluded n'est pas rejoué. Une réservation processing trop ancienne peut être reprise après PIPELINE_PROCESSING_STALE_SECONDS.
- Un HTTP 429 Gmail libère la réservation sans consommer de tentative et arrête uniquement l'entreprise concernée pour le passage.
- Un invalid_grant marque la connexion Gmail comme revoked. L'avis Telegram de révocation n'est enregistré qu'après livraison réussie.
- Le brouillon Gmail est créé avant la seconde écriture Sheet, mais son identifiant est immédiatement persisté afin d'éviter une recréation après panne.
- Les actions Telegram utilisent un jeton opaque, réservent atomiquement l'état sending/discarded et rendent les doubles clics sans effet métier répété.

3.4 Isolation multi-tenant et états opérationnels

Pour les routes utilisateur, l'entreprise est dérivée du JWT et rechargée depuis PostgreSQL ; elle n'est jamais choisie librement dans la requête. Les lectures Google Sheets sont recoupées avec le registre Neon du même company_id. Les connexions Gmail, Drive et Telegram sont liées au propriétaire interne de cette entreprise.

État	Source	Effet
En attente / Validée	PIPELINE_ALLOWED_COMPANY_IDS	Autorise ou non le traitement réel ; le dashboard ne reçoit jamais la liste complète.
Active / Suspendue	Champs suspended_at, suspended_until et suspension_reason	Une suspension bloque seulement le pipeline. Le dashboard reste accessible ; une date passée réactive automatiquement la sélection.
Prête / À compléter	Gmail + Sheet requis	Telegram n'est pas requis ; il fait passer le compteur de 2/3 à 3/3.

Communication administrateur
L'administration peut envoyer un message interne destiné à la bannière du dashboard. L'API utilisateur retourne uniquement le dernier message non lu ; elle n'expose ni liste ni pagination. L'audit administratif enregistre le type d'action, l'acteur et la date, jamais le corps du message.

4. Serveur et services externes

4.1 API FastAPI

L'application enregistre 42 opérations OpenAPI dans l'environnement de développement. En production, Swagger, ReDoc et le schéma OpenAPI sont désactivés ; les interfaces et les API restent servies sur la même origine afin de conserver une CSP connect-src 'self'.

Domaine	Opérations	Responsabilité
Admin	11	Session secondaire, identité, vue d'ensemble, entreprises, messages, suspension et incidents.
Dashboard	10	Statut opérationnel, bannière, indicateurs, emails, brouillons et actions utilisateur bornées.
Companies	5	Création du Sheet, entreprise courante, paramètres, validation et réparation d'accès.
Google Drive OAuth	4	Consentement drive.file, callback, état et déconnexion sans suppression du fichier.
Gmail OAuth	3	Consentement gmail.modify, callback et état.
Telegram	3	Code temporaire, statut et webhook protégé.
Auth	2	Inscription sur invitation et connexion.
System / Contact / Pipeline	4	Health, readiness, formulaire public et déclenchement cron.

4.2 Modèle PostgreSQL

Table	Rôle	Contraintes structurantes
users	Comptes SmartReply	Email unique ; mot de passe Argon2.
companies	Entreprise, Sheet, profil de réponse et suspension	Une entreprise par utilisateur ; sheet_id unique ; ton et mode de propriété contraints.
gmail_connections	OAuth Gmail	Une connexion par utilisateur ; token chiffré ; connected/revoked.
drive_connections	OAuth Drive	Une connexion par utilisateur ; scope drive.file ; token chiffré.
telegram_link_codes	Codes SR-XXXXX	Code unique, expiration 15 min, un pending par utilisateur.
telegram_connections	Liaison au chat	Utilisateur et chat uniques.
processed_emails	Idempotence et état du brouillon	Unicité entreprise/message, tentatives bornées, callback opaque unique.
company_activity_events	Transitions du dashboard	Unicité événement/traitement ; aucun contenu d'email.
admin_user_messages	Bannière interne	Ciblage entreprise, auteur interne, read_at.
admin_audit_events	Journal des actions admin	Types contraints ; aucun corps de message.

4.3 Migrations

Le schéma est construit par 12 migrations Alembic ordonnées. Une base neuve reçoit successivement les comptes, OAuth Gmail, entreprises, Telegram, fondations multi-tenant, registre de traitement, actions Telegram, analytics du dashboard, OAuth Drive, ton et profil de réponse, puis messages et suspensions administrateur. La tête actuelle est a9c4e7f2b6d1.

```
alembic upgrade head
alembic heads # résultat attendu : a9c4e7f2b6d1 (head)
```

4.4 Intégrations et droits

Service	Droit / protocole	Données échangées
Gmail	OAuth 2.0 — gmail.modify	Lecture ciblée, création du brouillon et envoi uniquement après action utilisateur.
Google Drive	OAuth 2.0 — drive.file	Création et gestion des fichiers créés ou ouverts par SmartReply.
Google Sheets	Compte de service writer	Onglets Emails et Brouillons du classeur de l'entreprise.
Gemini	Clé serveur — modèle par défaut gemini-2.5-flash	Corps tronqué, métadonnées utiles et profil d'entreprise borné ; réponse JSON structurée.
Telegram	Bot API + secret de webhook	Notification, résumé et callback opaque ; intégration facultative.
SMTP	SSL authentifié	Champs bornés du formulaire Contact vers le destinataire configuré.

4.5 Clés et configuration

Les secrets sont fournis par variables d'environnement. Le dépôt ne contient ni valeur de production ni fichier de credentials. Au démarrage en production, la configuration vérifie les champs obligatoires, l'origine HTTPS, le format PostgreSQL, le compte de service, les budgets positifs et la cohérence de Telegram. Les erreurs ne renvoient que les noms des variables invalides.

Groupe	Variables représentatives	Règle
Application	ENVIRONMENT, BASE_URL, DATABASE_URL	Production exige une origine HTTPS et PostgreSQL.
Authentification	JWT_SECRET, INVITATION_CODES	Inscriptions fermées si aucune invitation ; JWT signé côté serveur.
OAuth	GOOGLE_CLIENT_*, GOOGLE_DRIVE_CLIENT_*, TOKEN_ENCRYPTION_KEY	Secrets serveur ; refresh tokens chiffrés avant persistance.
Pipeline	CRON_SECRET, PIPELINE_ALLOWED_COMPANY_IDS, budgets	Secret obligatoire ; allowlist vide = aucun traitement réel.
Administration	ADMIN_ALLOWED_USER_IDS, ADMIN_ACCESS_CODE	Accès fermé si une valeur manque ; code de 8 caractères.
Anti-abus	REDIS_URL, RATE_LIMIT_HMAC_SECRET, CLIENT_IP_HEADER	Identités HMAC ; repli mémoire si Redis est indisponible.

5. Applications, hébergement et déploiement

5.1 Formes de l’application

Forme	Technique	Particularité
SaaS web	FastAPI + interfaces statiques	Une même origine sert les pages et l’API ; responsive, sans application native.
Pipeline	Endpoint 202 + tâche d’arrière-plan	Déclenché par cron externe ; budget de temps et verrou global PostgreSQL.
Bot Telegram	Webhook FastAPI	Actif seulement si token et secret sont tous deux configurés.
Console admin	Page statique + API protégée	Aucun lien depuis l’accueil ou la sidebar utilisateur.

5.2 Hébergement prévu par le dépôt

Élément	Service	Responsabilité
Application web	Render	Exécution Uvicorn, variables d’environnement, HTTPS public et journaux de processus.
Base relationnelle	Neon PostgreSQL	Schéma applicatif, transactions, verrou advisory et données multi-tenant.
Limites partagées	Redis / Render Key Value facultatif	Compteurs anti-abus atomiques entre instances.
Données métier	Google Drive / Sheets du client	Propriété du fichier pour les nouvelles entreprises ; compte de service writer.
Automatisation	Cron externe non fixé dans le code	Appel périodique protégé de /api/v1/pipeline/run.
Code et CI	GitHub / GitHub Actions	Dépôt, branches, pull requests et validation automatisée.

Limite de vérification
Ce document n’atteste pas la configuration effective des tableaux de bord Render, Neon, Google Cloud, Telegram ou Redis. Il décrit les exigences et comportements présents dans le dépôt au commit de référence.

5.3 Procédure de déploiement

Étape	Action	Contrôle
1	Créer/configurer Neon et le service Render sans lancer l'application incomplète.	Préparer DATABASE_URL et les secrets ; ne pas les stocker dans Git.
2	Définir ENVIRONMENT=production, BASE_URL HTTPS et les variables obligatoires.	Le démarrage doit échouer si une valeur critique est absente ou invalide.
3	Déployer la révision validée.	Le procfile exécute alembic upgrade head avant Uvicorn.
4	Vérifier /health et /ready.	/health confirme le processus ; /ready valide la configuration et SELECT 1.
5	Vérifier le durcissement HTTP.	Docs absentes, API no-store, CSP, X-Frame-Options, HSTS en HTTPS production.
6	Configurer les callbacks OAuth, le cron et éventuellement Telegram.	Callbacks construits depuis BASE_URL ; webhook Telegram enregistré seulement si activé.
7	Réaliser un parcours de test isolé.	Compte, entreprise, Drive, Gmail, Sheet, pipeline, brouillon et absence de mélange tenant.

5.4 Commande de démarrage

```
alembic upgrade head && uvicorn app.main:app --host 0.0.0.0 --port $PORT --no-proxy-headers
```

Le choix --no-proxy-headers empêche Uvicorn de faire confiance aux en-têtes proxy fournis par le client. CLIENT_IP_HEADER ne doit nommer qu'un en-tête remplacé par l'ingress ; le dépôt recommande CF-Connecting-IP sur le service Render public.

6. Qualité et exploitation

6.1 Tests et intégration continue

Niveau	Outil	Couverture actuelle	Commande
Services métier	unittest + mocks	Filtrage, IA, profils, pipeline, idempotence, Telegram et Drive.	python -m unittest discover -s tests -v
API	FastAPI TestClient	Auth, entreprises, dashboard, admin, sécurité, OAuth et erreurs.	Même suite
Isolation	SQLite en mémoire + scénarios tenant	Messages, emails, brouillons, actions Telegram et pipeline par entreprise.	Même suite
Interfaces	Tests statiques + Node VM	DOM sûr, routes, états, responsive déclaré et purge de session.	Même suite
Migrations	Alembic + base vierge	Chaîne jusqu'à la tête et création des tables V2 admin.	alembic heads
Style	Ruff	Imports et erreurs E, F, I sur le dépôt complet.	ruff check --select E,F,I .

La workflow GitHub Actions s'exécute sur les pull requests et les pushes vers main ou arena/**. Elle installe Python 3.11, les dépendances et Ruff, puis enchaîne lint, compileall, 288 tests et contrôle de la tête Alembic.

6.2 Santé, incidents et journalisation

- /health répond uniquement que le processus HTTP est vivant ; la réponse ne divulgue aucun environnement ni secret.
- /ready revalide la configuration et exécute SELECT 1 sur PostgreSQL ; un échec répond 503 avec status=not_ready.
- La console admin agrège les révocations, échecs, traitements bloqués, notifications bloquées et synchronisations Sheet en références internes sûres.

- Les logs du pipeline utilisent des identifiants internes et des compteurs ; les modèles de contexte marquent les secrets repr=False.
- Les erreurs HTTP non gérées renvoient un message générique et conservent les en-têtes de sécurité.

6.3 Sauvegardes et restauration

État constaté
Le dépôt ne contient actuellement ni tâche de sauvegarde PostgreSQL, ni procédure automatisée de restauration, ni règle de rétention. Les données métier sont réparties entre Neon, Gmail et les Sheets détenus par les clients ; leurs mécanismes fournisseur ne sont pas vérifiés ici. Toute politique d'exploitation doit donc être définie et testée en dehors du code actuel.

6.4 Mesures de sécurité présentes

Zone	Mesure
Comptes	Argon2id, JWT HS256 expirant, utilisateur rechargé depuis PostgreSQL sur les routes protégées.
OAuth	Scopes gmail.modify et drive.file ; state signé ; refresh tokens chiffrés Fernet et jamais renvoyés au navigateur.
Administration	JWT + ADMIN_ALLOWED_USER_IDS + ADMIN_ACCESS_CODE ; cookie HttpOnly, SameSite=Strict, Secure en production, durée 30 min.
Anti-abus	Limites login/register/contact/admin, clés HMAC sans email/IP en clair, Redis atomique et miroir mémoire.
HTTP	CSP stricte, no-referrer, nosniff, DENY framing, Permissions-Policy, no-store API et HSTS conditionnel.
Pipeline	X-CRON-SECRET, verrou PostgreSQL, allowlist, budgets, suspension indépendante et idempotence.
Telegram	Secret webhook fail-closed, callback opaque sans identifiant métier et vérification du chat/tenant à chaque clic.
Données admin	Aucun token, identifiant Gmail/Sheet/Telegram, corps d'email, brouillon ou profil texte dans les réponses d'administration.

6.5 Points d'exploitation restant à formaliser

- Définir, automatiser et tester la sauvegarde/restauration de PostgreSQL, ainsi que la rétention attendue.
- Choisir et documenter une supervision centralisée : alertes /ready, erreurs pipeline, révocations OAuth, saturation et échecs de migration.
- Fixer le fournisseur du cron, sa fenêtre d'exécution, ses retries et son alerte en cas d'absence d'appel.
- Ajouter un fichier de verrouillage des dépendances si la reproductibilité binaire prime sur les versions minimales déclarées dans requirements.txt.
- Conserver Redis pour une limitation cohérente sur plusieurs instances ; le repli mémoire est volontairement local et perdu au redémarrage.
- Exécuter périodiquement un test réel en environnement de staging Google avec deux entreprises distinctes, sans copier de données sensibles dans les rapports.
- Compléter les tests statiques d'interface par des parcours navigateur de bout en bout si l'équipe met en place un environnement UI dédié.

7. Organisation du code

Le dépôt suit une organisation par domaines fonctionnels. Les interfaces statiques restent séparées des services métier ; les opérations externes sont encapsulées dans des services et les frontières HTTP sont validées par Pydantic.

Dossier ou fichier	Contenu
app/main.py	Création FastAPI, routeurs, pages statiques, santé, documentation non-production et gestion d’erreur globale.
app/core/	Configuration, allowlists, IP fiable, rate limiting, documentation et en-têtes de sécurité.
app/db/	Base SQLAlchemy, sessions PostgreSQL et client Google Sheets.
app/modules/auth/	Inscription, invitations, Argon2, JWT et dépendance utilisateur.
app/modules/companies/	Entreprise, profil de réponse, validation, propriété du Sheet et suspension.
app/modules/gmail_oauth/ et drive_oauth/	Consentements séparés, chiffrement des tokens, états et révocation.
app/modules/pipeline_selection/	Éligibilité, équité, simulation/réel, budgets, suspension et verrou.
app/modules/pipeline_execution/	Orchestration isolée Gmail → Gemini → Sheets → brouillon → Telegram.
app/modules/processed_emails/	Registre transactionnel, dédoublonnage, tentatives et reprises.
app/modules/dashboard/	Agrégats Neon, recoupement Sheet, emails, brouillons, bannière et statut opérationnel.
app/modules/admin/	Triple contrôle d’accès, indicateurs, entreprises, incidents, messages, suspensions et audit.
app/home, dashboard, admin, contact, conditions, confidentialite	HTML, CSS et JavaScript natifs servis par FastAPI.
alembic/versions/	12 migrations ordonnées ; tête a9c4e7f2b6d1.
tests/	26 fichiers de tests et 288 cas automatisés.
scripts/	Configuration du webhook Telegram et réparation globale des permissions Sheets.
.github/workflows/tests.yml	Validation CI Python 3.11 sur push et pull request.

7.1 Conventions observées

- Une route protégée dérive toujours le tenant du JWT ou d’une relation interne vérifiée ; un company_id fourni par l’utilisateur ne suffit jamais.
- Les secrets vivent dans l’environnement, les refresh tokens sont chiffrés, et les réponses publiques utilisent des références internes préfixées lorsque nécessaire.
- Les migrations Alembic sont la seule source de création/évolution du schéma en déploiement ; le procfile les applique avant Uvicorn.
- Les écritures externes non transactionnelles sont entourées de marqueurs Neon afin de rendre les retries explicites et bornés.
- Le contenu venant des emails ou de l’administration est rendu avec textContent dans les interfaces ; aucun HTML métier non fiable n’est injecté.
- Les nouvelles fonctionnalités visibles portent un badge vert permanent « Nouveau » jusqu’à son retrait explicite du code.
- Les changements passent par une branche, une pull request, les tests, Ruff, compileall et la vérification Alembic avant fusion.

7.2 Documents et références liés

Document	Usage
SmartReply_Agent_SaaS_MVP_V1 (2).pdf	Référence de cadrage pour l'essence, la vision, le positionnement et les décisions produit.
README.md	Référence opérationnelle du dépôt : architecture, variables, procédures, matrices et endpoints.
/conditions/ et /confidentialite/	Cadre public du pilote, des responsabilités et du traitement des données.
OpenAPI hors production	Contrat HTTP généré depuis les schémas et routeurs FastAPI.
Migrations Alembic	Historique exécutable du modèle PostgreSQL.

7.3 Fiche de maintenance du présent document

Champ	Valeur
Titre	SmartReply Agent — Document technique
Version	1.0
Date	5 novembre 2026
Auteur	Koffa Jean Agoudavi
Dépôt analysé	koffaJeanAgoudavi-dev/smartreply-backend
Commit de référence	554f02221332880d2045d6d544c89095a426ca26
Tête Alembic	a9c4e7f2b6d1
Validation locale	288 tests réussis ; Ruff E/F/I ; JavaScript et chaîne Alembic contrôlés dans le dépôt.

Règle de mise à jour
Lors d'une mise à jour, recalculer les opérations API, les tables, les migrations et les tests ; vérifier les variables de production ; puis distinguer clairement ce que le dépôt prouve de ce qui dépend d'une console fournisseur externe.